DO NOW — RECALL

# Hashing and applications

Name: _____ Class: _____ Date: _____

**1. What is the encryption key in symmetric encryption?**

**2. What is the role of a public key?**

**3. Why encrypt a message in transit?**

**4. Compute $10110110_2$ AND $00001111_2$, then hexadecimal.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

A shared secret used to encrypt and decrypt.

### 2. Model answer

Can be distributed for encryption or verification according to the scheme.

### 3. Model answer

To protect its content from unintended readers.

### 4. Model answer

$00000110_2 = 06_{16}$; high four bits masked away.

### Vocabulary — know these terms

hash — Fixed-size digest of arbitrary input.

collision — Two inputs producing the same digest.

salt — Random value mixed into password hashing.

KDF — Key/password derivation function.

integrity — Confidence that data has not changed.

NEW LEARNING

# Read, annotate and explain

## Understand

A cryptographic hash transforms an arbitrary input into a fixed-size digest. It is designed to make reversing the digest infeasible, not impossible under all circumstances.

## Trace

A message digest supports integrity checking when the reference hash is trusted. Hash collisions can exist because outputs have finite length.

## Evaluate

Passwords should be processed using a salted, purpose-built slow password-hashing scheme, not simply a fast general-purpose hash and never reversible encryption.

**Explain one step in your own words, using the correct technical terms.**

# New Learning · original lesson diagrams

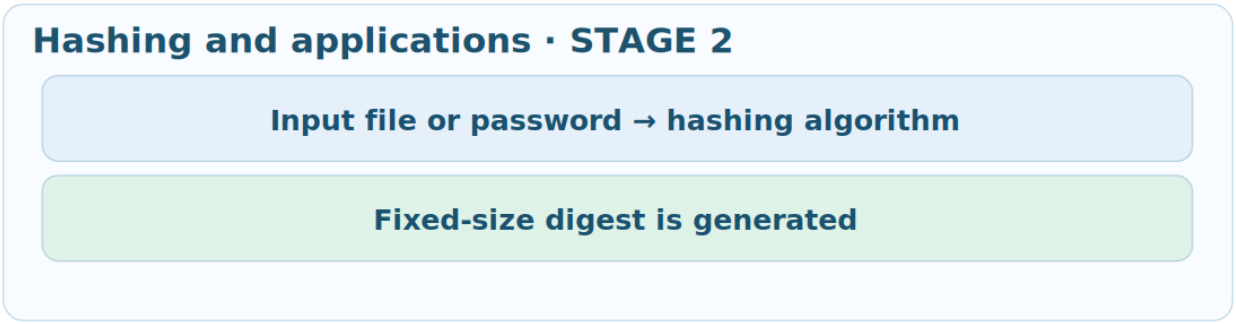
Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**



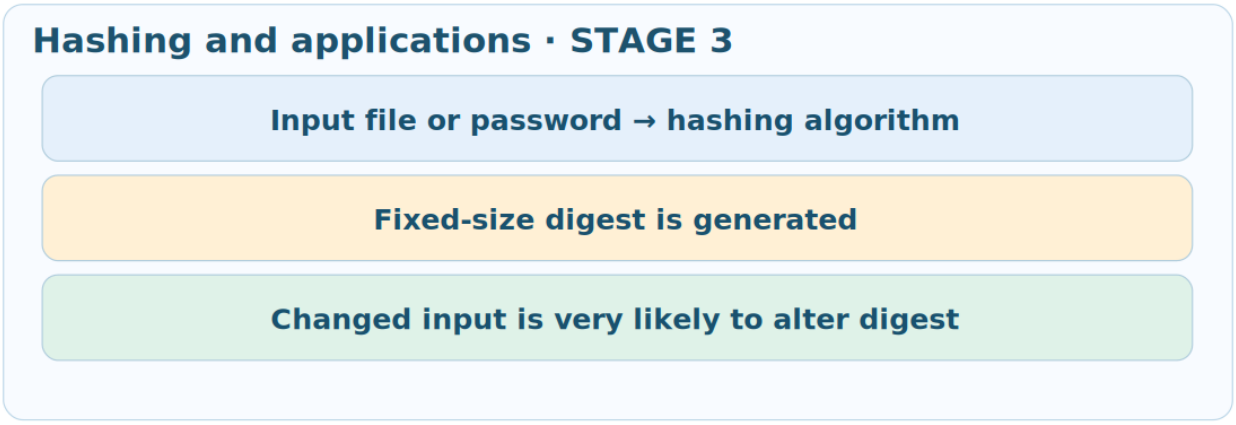

Hashing and applications · STAGE 1: Input file or password → hashing algorithm

**New Learning**




Hashing and applications · STAGE 2: Input file or password → hashing algorithm then Fixed-size digest is generated

**New Learning**

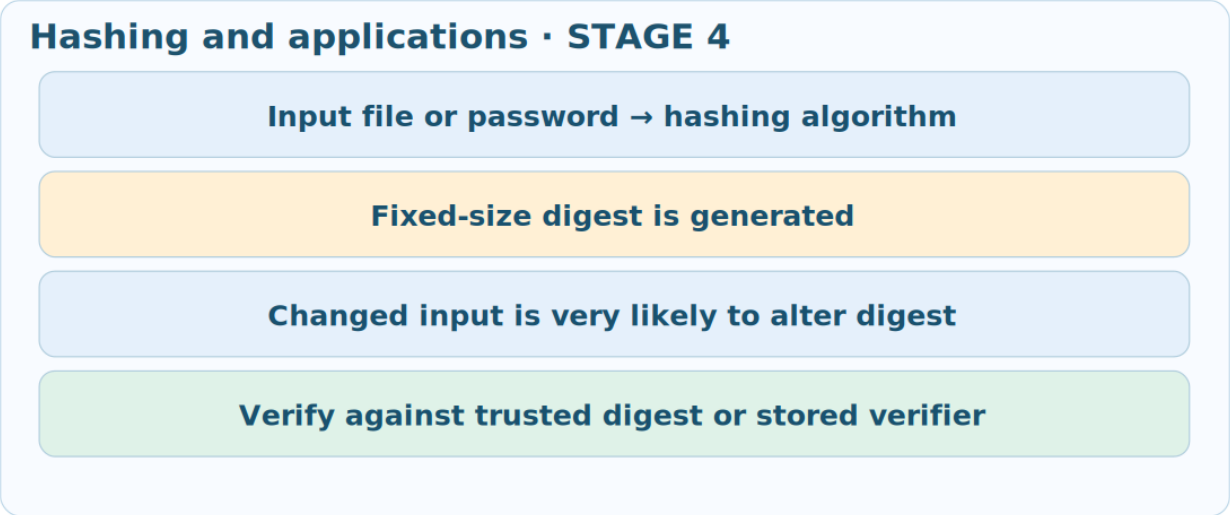



Hashing and applications · STAGE 3: Input file or password → hashing algorithm then Fixed-size digest is generated then Changed input is very likely to alter digest

# New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**New Learning**




Hashing and applications · STAGE 4: Input file or password → hashing algorithm then Fixed-size digest is generated then Changed input is very likely to alter digest then Verify against trusted digest or stored verifier

I DO — WORKED EXAMPLE

# Follow the worked method

### A downloaded file's trusted SHA-256 checksum differs from the computed checksum. What can you conclude?

### Predict first

The file contents or expected reference differ: the integrity check fails.

### Show method

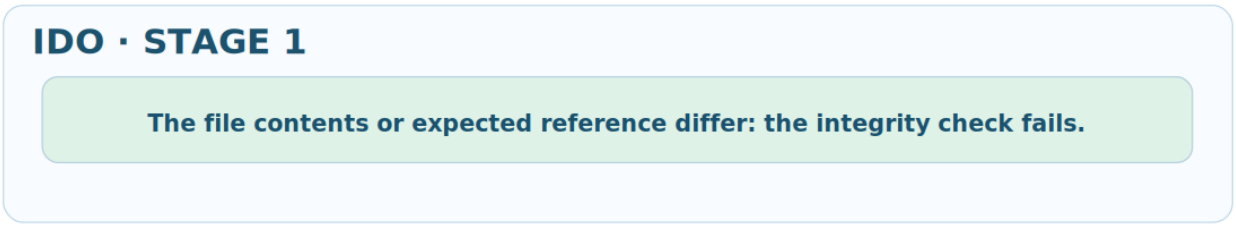
The mismatch does not by itself reveal whether alteration was accidental or malicious.

# I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

**A downloaded file's trusted SHA-256 checksum differs from the computed checksum. What can you conclude?**

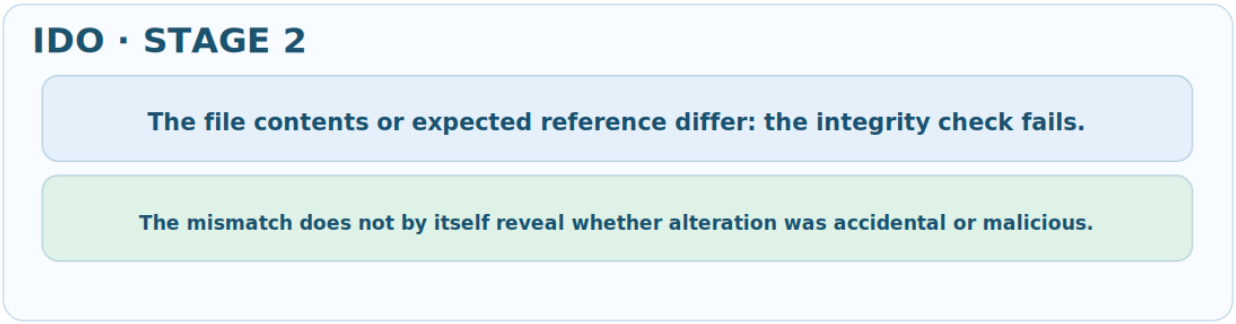
The file contents or expected reference differ: the integrity check fails.




IDO · STAGE 1: The file contents or expected reference differ: the integrity check fails.

**A downloaded file's trusted SHA-256 checksum differs from the computed checksum. What can you conclude?**

The file contents or expected reference differ: the integrity check fails.




IDO · STAGE 2: The file contents or expected reference differ: the integrity check fails. then The mismatch does not by itself reveal whether alteration was accidental or malicious.

WE DO — GUIDED PRACTICE

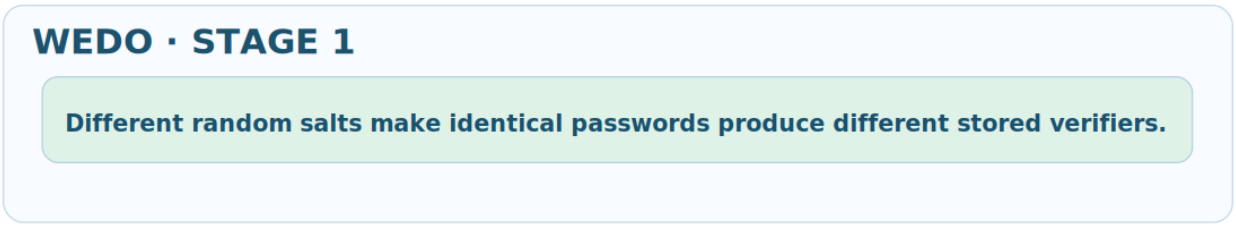
# Try → Discuss → Improve

# We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### Two users choose the same password. How can salts improve their stored password verifiers?

Different random salts make identical passwords produce different stored verifiers.




WEDO · STAGE 1: Different random salts make identical passwords produce different stored verifiers.

# WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

### Guided 1: Predict first Different random salts make identical passwords produce different stored verifiers.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

Define cryptographic hashing and distinguish it from reversible encryption. [6]

### Question 2

Explain how a trusted checksum helps verify a software download and give one limitation. [6]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

Explain salted password hashing and why storing raw passwords or just fast unsalted hashes is weaker. [6]

EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**If a password is hashed, the server simply decrypts that hash during login.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

## GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

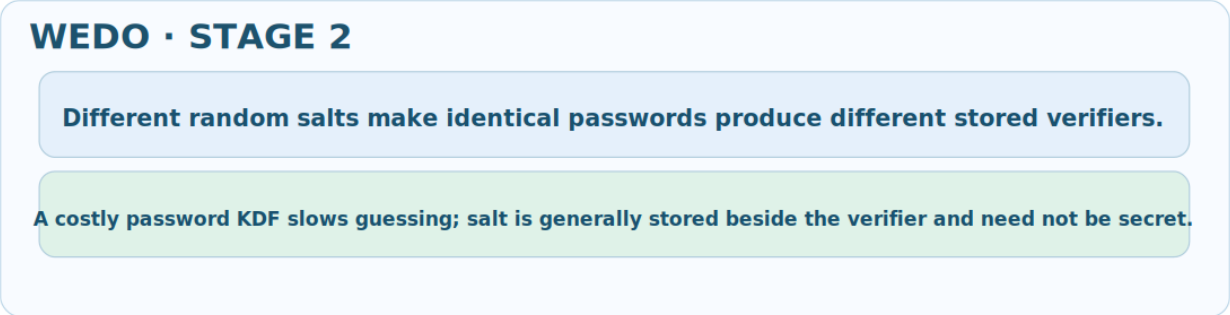
Next step: revisit the model or a relevant revision-guide section if needed.

# We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

### Two users choose the same password. How can salts improve their stored password verifiers?

Different random salts make identical passwords produce different stored verifiers.




WEDO · STAGE 2: Different random salts make identical passwords produce different stored verifiers. then A costly password KDF slows guessing; salt is generally stored beside the verifier and need not be secret.

# WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

### Guided 1 — original CHECK

Show method

A costly password KDF slows guessing; salt is generally stored beside the verifier and need not be secret.

IMPROVE: Explain one correction to your own reasoning.